Complete Repository Feature Audit
GoPAL-AI | Repository snapshot: 12 October 2026
Repository: [jamlove1318-lab/GoPAL-AI]https://github.com/jamlove1318-lab/GoPAL-AI
Branch: main
Scope: Static repository-tree and README review. This is not a completed build, device test, or proof that every documented feature is reachable.
Evidence rule: A file, asset, workflow, contract or README claim proves that an implementation or design artifact exists-not that the feature is wired into the shipped app. Treat conclusions as a map for verification, not as a declaration that code is unused.

1. Repository snapshot

The recursive Git tree contained 705 paths at the audit snapshot. The README describes GoPAL-AI as an autonomous language-learning companion and living sanctuary, with multi-turn Socratic roleplay, world continuity and multiple domain engines.

Root and supporting areas

Path	Role / what to verify
App.tsx, index.ts	App bootstrap and actual screen/navigation entry path
app.json, eas.json, package.json	Expo metadata, build profiles, dependencies and scripts
tsconfig.json, Babel/Metro/Tailwind config, global.css	TypeScript, bundling and styling setup
src/	Main application source: engines, characters, components and archive
assets/	Icons, Cassidy references, Emerald Valley environment art, animation and media
docs/	Master specifications, workflows, implementation trackers, Cassidy production and world/learning/audio documentation
.github/workflows/	APK build, Cassidy production, remote art, world asset acquisition, repair, validation and typecheck automation
scripts/, tools/, bin/, factory/, jobs/	Development and asset-production tooling
contracts/	Expected integration boundaries and validation contracts
supabase/	Backend schema/policy candidates; inspect before assuming cloud behavior
artifacts/	Generated evidence, reports, packages and validation outputs
src/archive/	Explicitly preserved legacy material

Path caution: Older audit notes refer to top-level app/, engine/, store/ and data/. The current recursive tree instead foregrounds src/ and does not list those as root directories. Use the current branch as the source of truth.

2. App entry, build and delivery

- App.tsx, index.ts - PRESENT: bootstrap candidates. Trace the real render and navigation chain before assigning connection status.
- Expo / TypeScript configuration - PRESENT: app.json, eas.json, package.json, tsconfig.json, Babel/Metro/Tailwind configuration and global.css.
- Build automation - PRESENT: .github/workflows/build-release-apk.yml. The README documents Yarn install, npx tsc --noEmit, Expo start and EAS Android/iOS build profiles.
- Not established by this audit: successful current build, fresh-install navigation,

physical-device stability, offline restart, or store-release readiness. No build was run here.
- Next checks: clean install from the declared package manager; typecheck and configured lint; Expo export/build; fresh-install smoke test; restart/offline test. Check .env.example and never commit live secrets.

3. Experience director, events and domain engines

Source location	Identifiable responsibility	Connection status
src/engines/index.ts, testRunner.ts	Engine exports and test orchestration candidates	Verify imports/call sites
src/engines/director/experienceDirector.ts	Experience/intent selection and coordination	Verify instantiated runtime path
src/engines/events/eventBus.ts	Shared domain event coordination	Verify event producers/consumers
src/engines/character/characterEngine.ts	Character domain	Verify screen/engine call path
src/engines/creation/creationStudio.ts	Learner creation experiences	Verify save/reopen route
src/engines/knowledge/knowledgeEngine.ts	Knowledge/exhibit concepts	Verify Museum connection
src/engines/journey/journeyEngine.ts	Personal chronology and growth	Verify actual Journey route
src/engines/economy/economyEngine.ts, quest/questEngine.ts	Optional goals, rewards and economy	Verify persistent state and UI

Intended architecture: learner intent and context -> director -> learning/character/world activity -> typed event -> memory and state -> meaningful change on a later visit. Presence of engine files is not proof they are all instantiated or coordinated.

4. Learning and tutor systems

Location: src/engines/learning/ contains:
- adaptiveConversationEngine
- contextualLanguageLearningEngine
- languageCapabilityEngine
- learningEvaluationEngine
- learningNeedEngine
- residentConversationContinuityEngine
- wordBankAnswerEngine
- worldLearningFlowEngine, worldLearningIntegrationEngine, worldLearningOutcomeEngine, worldLearningResponseEngine, worldLearningScenarioEngine

Tutor entry candidate: src/engines/tutor/tutorEngine.ts.
The README describes multi-turn Socratic roleplay, contextual immersion, pitch-accent shadow practice, calligraphy/stroke order, dialogue transcripts, phonetics and audio drills.
Expected loop: infer learner need/ability -> choose scenario -> roleplay -> evaluate -> adapt next prompt -> save transcript/vocabulary -> revisit later -> show growth in Journey.

Integration questions: Does evaluation change difficulty? Are recurring errors revisited? Are transcripts and audio reachable from active navigation? Do resident and tutor dialogue share one learner model? Confirm with source call paths and tests.

5. Cassidy: character, autonomy and presentation

Character source: src/characters/ includes Cassidy identity/runtime candidates such as canonical reference intake, design/master, production pipeline, context, asset contracts/registry, integration gate, production spec, reference package, renderer contract/controller, runtime model contract, visual design/resolver and NPC design.

Behavior engines: src/engines/cassidy/ contains action/autonomy, character runtime/system, experience coordination, interaction/lifecycle, learning reaction, life director/state, meaningful/narrative experience, perception, personality, presence, presentation, runtime bridge/session, scene anchor, welcome, world presence and verification modules.

Presentation candidates: src/components/Cassidy2DProductionRenderer.tsx, CassidyCharacter.tsx, LivingCompanion.tsx, LivingWorldPulse.tsx, AmbientBackground.tsx.

Production evidence: canonical reference imagery, artifacts/cassidy/ acceptance/package/checksum/production/validation files, and .github/workflows/cassidy-production.yml. This shows a production process; it does not by itself prove a live 3D renderer in the app.

Risks / acceptance criteria

- Multiple life/state/autonomy/presence/presentation layers can overlap. Define one owner per responsibility and typed contracts.
- Use one canonical identity and asset manifest, deterministic tests and graceful 2D fallback.
- Cassidy should remember relevant experiences honestly, react to learning/world context, and keep autonomy bounded and respectful.
- Relationship should be shown through behavior and shared moments-not a visible affection score.

6. Emerald Valley and living-world systems

Assets and specs: assets/world/emerald-valley/ contains environment/atmosphere resources, including HDRI assets. Documentation includes docs/EMERALDVALLEYWORLDMASTERSPEC.md and real asset acquisition reports.

World engines in src/engines/world/ include:

- World/language-world engine and state; environment and contextual world events; continuity.
- Destination arrival/departure/encounter/experience/opportunity/resident systems.
- Living world clock, objects store, reactor, runtime and simulation.
- NPC registry/runtime; offline resident behavior; persistent world state.
- Place presentation; resident action, ambient behavior, animation, consequences, context reactions, relationships, relationship memory, routes, routines, stories and story opportunities.
- Activity content/mechanics/primitives; animation/audio contracts and runtime; crafting, inventory, investigation, discovery progression, hotspots, external asset registry/binding, landmarks and integrity.

Related documentation: world audio, source policy, external art/animation pipelines, mini-game catalogs, learning variety, batch integration, verification and current progress.
Intended loop: clock/weather/season/location/history -> event director -> scene/resident/activity -> language opportunity -> learner action -> event and memory -> changed return visit.
Risks to verify: duplicated state, circular dependencies, hard-coded transitions, schema drift, unhandled offline behavior and assets that exist but are not bound to a reachable screen.

7. Memory, Journey, economy, audio and creation

- Memory: src/engines/memory/memoryEngine.ts plus resident/world memory modules. Separate session context, durable learner facts, relationship memories, world canon, event history and private goals.
- Journey: src/engines/journey/journeyEngine.ts; README describes milestones, narrative chapter reviews and then-vs-now speech comparisons.
- Economy/quests: src/engines/economy/economyEngine.ts, src/engines/quest/questEngine.ts; README mentions daily quests, personal challenges, rare seeds, fertilizer and souvenirs.
- Knowledge/Museum: src/engines/knowledge/knowledgeEngine.ts; README describes postcards, learner creations, cultural keepsakes and canonical story memories.
- Audio: src/engines/audio/audioEngine.ts and world audio modules; README describes a four-channel ambience mixer and world-radio stations.
- Creation: src/engines/creation/creationStudio.ts; verify save/reopen, export, privacy and attribution.
- Cloud/storage: supabase/ exists, but actual schema, app wiring, row-level security and sync correctness must be inspected. Presence is not proof of secure cloud persistence.

8. README-described product surfaces

Surface	README-described capabilities	Evidence still needed
Home / Living Sanctuary	Time-aware greeting, away recap, continuity cards, festivals, natural-language World DJ	Active route, event source, user intent routing
Study	Bonsai, matcha workshop, calligraphy, soundscape, pitch practice	Reachable actions, accessibility, saved outcomes
World Map	Emerald Valley locations, familiarity, resident bios, transitions, lore, ema wish ritual	Location routing, persistent discovery and resident state
Journey	Milestones, chapter reviews, speech-growth comparison, transcripts	Real learner data and privacy controls
Museum	Postcards, creations, keepsakes, story memories	Save/reopen and deletion/export
Settings	Runtime diagnostics and engine/storage verification	Actual on-device checks and understandable failures

These are product claims documented in README; each should be confirmed against active screens and automated tests.

9. Duplicated, unfinished or potentially orphaned work

1. Overlapping world systems: continuity, destination experience, resident presentation,

memory and event coordination need clear ownership.

2. Cassidy lifecycle breadth: life/state/autonomy/presence/presentation/runtime layers need

one documented state model and integration boundary.

3. Specs versus runtime: production checklists, acceptance artifacts and asset pipelines do

not prove live rendering or feature reachability.

4. Status drift: reconcile documentation trackers and progress reports with current source

and commit.

5. Legacy: src/archive/learning/WorldActivityExperienceModal.legacy.tsx is explicitly

archived; do not delete without reference and replacement checks.

6. Potential orphan candidates: only label unused after tracing static imports, dynamic

imports, registries, routes, workflows and runtime behavior.

10. Recommended verification order

1. Freeze the audited commit SHA; install dependencies and run typecheck/lint/build.

2. Inventory active routes from the real navigation tree.

3. For each screen, trace action -> component -> engine/service -> storage -> rendered

consequence.

4. Test onboarding/re-entry, Study, roleplay, World Map, transcripts, memory, quests,

Journey, Museum, Settings and offline restart.

5. Verify Supabase policies, privacy controls, migrations, sync conflicts and

deletion/export.

6. Tie generated artifacts to commit, input hashes and reproducible validation.

7. Publish a feature matrix with four states: CONNECTED + tested, CONNECTED but untested,

IMPLEMENTED/ORPHANED, SPEC/DOC/ASSET ONLY.

Preservation rule

Nothing gets deleted until its purpose is verified and cleanup is agreed. Prefer archive,
deprecate or refactor additively first.